Master 2 Intelligence Artificielle – Systèmes multi-agents

TP / Projet sous Jade

Système multi-agents BDI de réservation de taxis

Benhamaid Mohamed Amine 12512882
Choayb BOUMAZA 12513241

Enseignante : Pr. Nadia Kabachi

9 novembre 2026

Table des matières

1 Installation de Jade sous Windows **1**

2 Découverte de la plateforme **2**

2.1 Les agents système 2

2.2 L'exemple `bookTrading` 2

3 Premier agent et communication **3**

3.1 HelloAgent 3

3.2 Échange de messages ACL 4

4 Modélisation BDI du système de taxis **5**

4.1 Environnement 5

4.2 Croyances, désirs et intentions 5

4.3 Le client : un automate d'intentions 5

4.4 Le taxi : un plan d'étapes 6

4.5 Protocole d'interaction 6

5 Scénarios et résultats **8**

5.1 Scénario nominal : compromis prix / attente 8

5.2 Concurrence entre clients, et mode réservation 8

5.3 Négociation et révision des désirs 8

5.4 Carburant, profils, covoiturage et trafic 8

5.5 Simulation continue et comparaison avec un dispatcher 9

6 Conclusion **10**

Objectif. Ce TP présente d'abord la plateforme JADE (Java Agent DEvelopment Framework) : installation, agents système et premiers échanges de messages. Il l'utilise ensuite pour concevoir une application suivant l'architecture BDI (*Belief – Desire – Intention*). Parmi les trois sujets proposés, nous avons choisi la **réservation de taxis** : dans une ville simulée, des clients cherchent un transport et des taxis proposent leurs services. Chaque agent agit de façon autonome, selon ses propres objectifs.

1 Installation de Jade sous Windows

Environnement. Windows 11 Pro, JDK 24 (JADE demande au minimum Java 8) et IntelliJ IDEA. Nous avons utilisé JADE 4.6.0, la version disponible au téléchargement. Elle est compatible avec la 4.5.0 citée dans l'énoncé.

Étapes.

1. Télécharger `JADE-all-4.6.0.zip` sur `http://jade.tilab.com` et le décompresser. On obtient quatre dossiers : `JADE-bin` (bibliothèque `lib/jade.jar`), `JADE-doc`, `JADE-examples` et `JADE-src`.
2. Vérifier Java dans un terminal : `java -version`.
3. Dans IntelliJ, créer un projet Java. Copier `jade.jar` dans un dossier `lib/` du projet, puis l'ajouter dans *File > Project Structure > Libraries > + Java*. Le jar se trouve alors dans le

classpath.

4. Créer une configuration *Run > Edit Configurations > + Application* avec la classe principale `jade.Boot` et les arguments `-gui`. Pour démarrer des agents, on ajoute : `-gui -agents "nom:paquet.Classe(arg1,arg2)"`.

Sans IDE, on peut aussi lancer la plateforme en ligne de commande :

```
java -cp lib\jade.jar jade.Boot -gui
java -cp "lib\jade.jar;out" jade.Boot -gui -agents "hello:firstAgent.HelloAgent(Amine)"
```

Sous Windows, le séparateur du classpath est le point-virgule (`;`), alors que c'est `:` sous Linux. Pour éviter de taper ces commandes, le projet fournit un script `run.ps1` qui compile tout le code puis lance un exercice ou un scénario (`.\run.ps1 nominal`). La plateforme écoute par défaut sur le port 1099 : si une instance tourne déjà, la suivante échoue avec *Cannot bind server socket to localhost port 1099*.

2 Découverte de la plateforme

2.1 Les agents système

La commande `jade.Boot -gui` démarre le *Main-Container*, qui contient trois agents système :

— **AMS** (*Agent Management System*) : gère le cycle de vie des agents et leur nommage (AID). C'est l'autorité de la plateforme.

— **DF** (*Directory Facilitator*) : un annuaire de type « pages jaunes ». Les agents y publient les services qu'ils offrent, et d'autres agents peuvent les y chercher.

— **RMA** (*Remote Monitoring Agent*) : l'interface graphique. Elle permet de créer, suspendre et tuer des agents, et de lancer les outils de débogage.

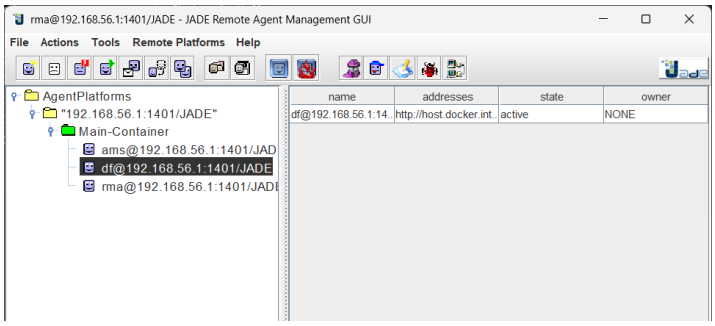


FIGURE 1 – Fenêtre du RMA : Main-Container avec l'AMS, le DF et le RMA.

Depuis le menu *Tools* du RMA, on accède aux autres outils vus en cours :

— le **DummyAgent**, pour composer et envoyer un message ACL à la main ;

— le **Sniffer**, qui dessine les messages échangés entre agents sous forme de diagramme de séquence ;

— l'**Introspector**, qui montre la file de messages et les comportements actifs d'un agent ;

— l'interface du DF, qui affiche les services enregistrés.

2.2 L'exemple `bookTrading`

Nous avons fait fonctionner l'exemple `examples.bookTrading` fourni avec JADE, puis étudié son fonctionnement.

— Chaque `BookSellerAgent` s'inscrit au DF sous le service `book-selling` et ouvre une petite fenêtre pour ajouter des livres (titre, prix) à son catalogue.

— Le `BookBuyerAgent` reçoit en argument le titre recherché. Un `TickerBehaviour` interroge régulièrement le DF pour trouver les vendeurs, puis lance la négociation.

— La négociation suit le *Contract Net* : `CFP` (titre) → `PROPOSE` (prix) ou `REFUSE` → `ACCEPT_PROPOSAL` au vendeur le moins cher → `INFORM` (vente conclue) ou `FAILURE` (livre déjà vendu).

Pour les démonstrations, nous avons légèrement adapté notre copie de l'exemple. Les livres d'un vendeur peuvent être passés en arguments (`bkVendeur1:...BookSellerAgent(Le-Petit-Prince,25,Germinal,12)`) et la période de recherche de l'acheteur, fixée à 60 s dans l'original, est réglable (6 s ici). Le Sniffer est lancé directement avec la liste des agents à observer (`sniffer:jade.tools.sniffer.Sniffer(bk*)`), pour que tous les messages soient capturés dès le départ. Sur la figure 2, l'acheteur interroge le DF (REQUEST/INFORM), envoie un CFP aux deux vendeurs, reçoit deux offres (25 et 19), accepte la moins chère, puis reçoit la confirmation de vente (INFORM).

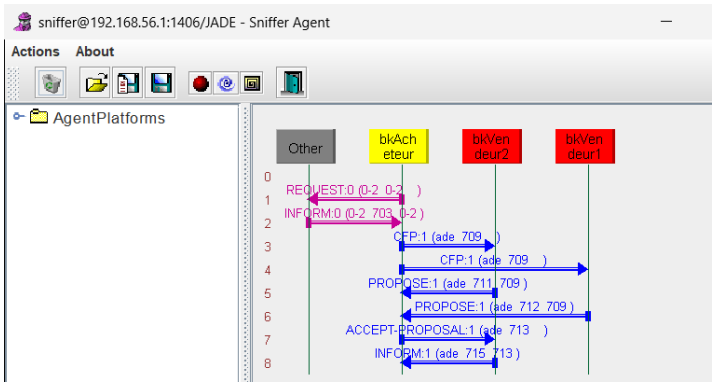


FIGURE 2 – `bookTrading` observé dans le Sniffer : recherche au DF, CFP aux deux vendeurs, deux PROPOSE, ACCEPT_PROPOSAL au moins cher, INFORM.

Cet exemple contient déjà presque tout ce dont notre projet aura besoin : annuaire DF, comportements, performatifs FIPA et identifiants de conversation. Il nous a servi de base.

3 Premier agent et communication

3.1 HelloAgent

Un agent JADE est une classe qui hérite de `jade.core.Agent`. Sa méthode `setup()` est appelée au démarrage, et `takeDown()` juste avant sa destruction. Les arguments de lancement se récupèrent avec `getArguments()`.

```
package firstAgent;
import jade.core.Agent;

public class HelloAgent extends Agent {
    protected void setup() {
        System.out.println("Bonjour ! Je suis l'agent " + getLocalName()
                + " (AID complet : " + getAID().getName() + ")");
        Object[] args = getArguments();
        if (args != null)
            for (Object a : args) System.out.println(" - " + a);
        doDelete(); // travail terminé
    }
    protected void takeDown() {
        System.out.println("Au revoir, l'agent " + getLocalName() + " se termine.");
    }
}
```

```
Bonjour ! Je suis l'agent hello (AID complet : hello@192.168.56.1:1099/JADE)
hello : mes arguments sont :
  - Amine
  - TP-JADE
Au revoir, l'agent hello se termine.
```

L'AID complet a la forme `nom@plateforme` : le nom local est unique dans la plateforme.



FIGURE 3 – Lancement de HelloAgent : l'agent affiche son AID et ses arguments, puis se supprime.

3.2 Échange de messages ACL

Un message est une instance de `ACLMessage`, créée avec un performatif FIPA (INFORM, REQUEST, CFP...). Le `SenderAgent` envoie un INFORM puis un REQUEST au `ReceiverAgent`. Celui-ci répond avec `createReply()`, qui remplit automatiquement le destinataire, l'identifiant de conversation, le protocole et le champ *in-reply-to*. Il ne reste qu'à fixer le performatif et le contenu.

```java
// émetteur
ACLMessage inform = new ACLMessage(ACLMessage.INFORM);
inform.addReceiver(new AID("receiver", AID.ISLOCALNAME));
inform.setContent("Bonjour, je suis " + getLocalName());
inform.setProtocol("information");
send(inform);

// récepteur (dans un CyclicBehaviour)
ACLMessage msg = receive();
if (msg == null) { block(); return; } // endort le comportement jusqu'au prochain message
ACLMessage reply = msg.createReply();
if (msg.getPerformative() == ACLMessage.REQUEST) {
    reply.setPerformative(ACLMessage.INFORM);
    reply.setContent("Il est " + LocalTime.now());
} else {
    reply.setPerformative(ACLMessage.CONFIRM);
    reply.setContent("Message bien reçu");
}
send(reply);
```

```
[sender] INFORM envoyé : Bonjour, je suis sender
[sender] REQUEST envoyé : heure
[receiver] reçu INFORM de sender : Bonjour, je suis sender
[receiver] reçu REQUEST de sender : heure
[sender] réponse reçue (CONFIRM, conv=salutation) de receiver : Message bien reçu, bonjour sender
[sender] réponse reçue (INFORM, conv=demande-heure) de receiver : Il est 12:14:13
```

Le couple `receive()` / `block()` est essentiel. Un comportement qui ne trouve pas de message se met en sommeil au lieu de consommer du CPU, et JADE le réveille à l'arrivée du message suivant. Avec un `MessageTemplate`, on ne lit que les messages d'un protocole ou d'une conversation donnés.

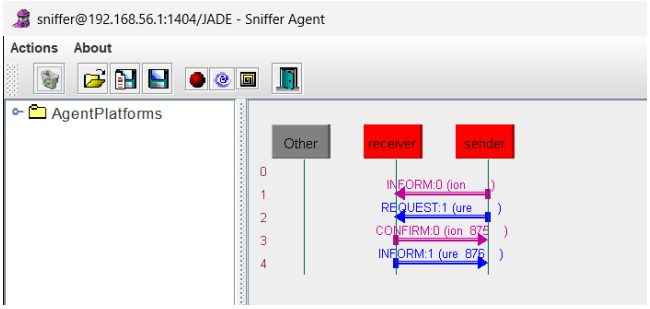


FIGURE 4 – Sniffer : INFORM puis REQUEST du sender, réponses CONFIRM et INFORM du receiver construites avec `createReply()`.

4 Modélisation BDI du système de taxis

4.1 Environnement

La ville est une grille 10×10 dont chaque case a des coordonnées (x, y). Deux stations-service se trouvent en $(0, 0)$ et $(9, 9)$. Le centre-ville ($3 \leq x \leq 6$, $4 \leq y \leq 5$) est embouteillé : y entrer coûte 3 ticks au lieu d'un. Le temps avance par *ticks* (400 ms par défaut), et un taxi parcourt une case par tick sur route fluide. Ses itinéraires sont calculés par l'algorithme de Dijkstra sur ce temps de parcours, et chaque case parcourue consomme une unité de carburant. Le prix d'une course est fixé par le taxi :

$$prix = (priseencharge + prix/km \times d_{Manhattan}(client, destination)) \times m \times r$$

où m est le multiplicateur dynamique et $r = 0{,}7$ en covoiturage ($r = 1$ sinon).

Pas de dispatcher central. Il n'existe que deux classes d'agents décisionnels, `ClientAgent` et `TaxiAgent`, et le DF sert d'annuaire. Les paramètres (position, destination, budget, tarif, carburant, profil) sont passés en arguments de lancement. Un `DispatcherAgent` existe aussi, mais uniquement pour comparer les deux approches (section 5.5).

4.2 Croyances, désirs et intentions

	Client	**Taxi**
Croyances	position, destination, budget, attente maximale, offres reçues, nombre d'échecs	position, état, carburant, tarif, gains, passagers à bord, demandes observées, offres en attente
Désirs	arriver en payant peu *et* en attendant peu, avec un poids qui dépend du profil. Désirs révisés après deux échecs	gagner de l'argent (utilité de chaque course) tout en restant opérationnel (carburant, risque de panne)
Intentions	chercher, choisir, négocier, attendre, voyager, relancer, abandonner	proposer ou refuser, aller chercher, transporter (y compris en covoiturage), se ravitailler, se repositionner

TABLE 1 – Modèle BDI des deux agents.

4.3 Le client : un automate d'intentions

Les intentions du client sont codées par un `FSMBehaviour` JADE. Chaque état est un comportement qui renvoie, en se terminant, l'événement qui choisit la transition suivante. Chaque transition remet l'état d'arrivée à zéro, pour qu'il reparte sur une situation propre.

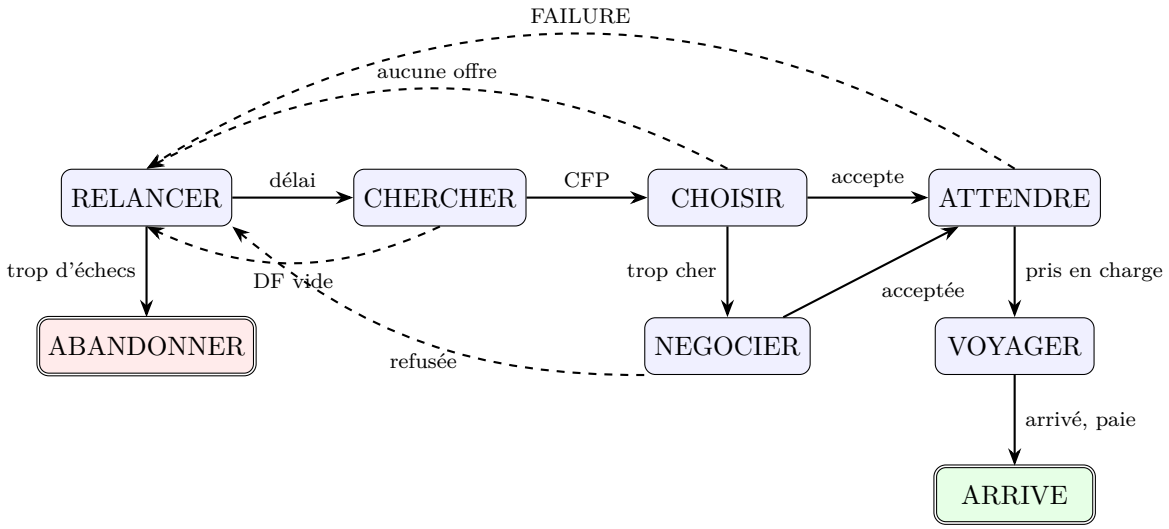


FIGURE 5 – Automate des intentions du client (`FSMBehaviour`). En pointillés : les échecs, qui renvoient à RELANCER.

Délibération. Le client écarte les offres dont le prix dépasse son budget ou dont l'attente dépasse son maximum. Parmi les offres restantes, il retient celle qui minimise

$$coût perçu = prix + w \times attente, \qquad w = 2{,}0\ (pressé),\ 0{,}5\ (normal),\ 0{,}1\ (économe).$$

Si aucune offre ne convient mais que l'une d'elles coûte au plus 1,4× le budget, le client **négocie** : il envoie une contre-offre égale à son budget. Après deux relances infructueuses, il **révise ses désirs** : son attente maximale est multipliée par 1,5 et il accepte de payer jusqu'à 120 % de son budget. Au-delà de son nombre maximal de relances, il abandonne.

4.4 Le taxi : un plan d'étapes

L'intention du taxi est un **plan**, c'est-à-dire une liste d'étapes `PRISE`, `DEPOSE`, `STATION` et `REPOS` qu'un comportement exécute case par case, à chaque tick. L'état affiché (LIBRE, REPOSITIONNEMENT, VERS_CLIENT, EN_COURSE, RAVITAILLEMENT, PANNE) n'est pas stocké : il se déduit du plan et des croyances. Cette représentation permet le covoiturage : pour prendre un second client, il suffit d'insérer sa prise en charge avant la dépose du premier passager.

Quand il reçoit une demande, le taxi délibère :

1. S'il est occupé, il refuse. S'il n'a qu'un passager, il évalue un **covoiturage**, accepté si le détour reste ≤ 3 ticks.
2. Si son carburant ne couvre pas approche + course + retour à la station la plus proche, il refuse et part se ravitailler.
3. Il calcule le prix avec un **tarif dynamique** : $m = 1 + 0{,}1\,(n-1)$ (plafonné à 1,5) si $n \geq 2$ clients *distincts* l'ont sollicité dans les 15 derniers ticks ; $m = 0{,}9$ après une longue inactivité.
4. Il calcule l'utilité de la course : $U = prix - 0{,}3 \times cases - risque$, avec une pénalité de risque si le carburant restant tombe sous 30 %. Si $U \leq 0$, il refuse (`pas-rentable`).

Un taxi libre et inactif se **repositionne** vers le barycentre des demandes qu'il a observées.

4.5 Protocole d'interaction

Les échanges suivent le *Contract Net* (protocole `taxi-cnp`). Les contenus sont codés en FIPA-SL avec une ontologie `TaxiOntology`, construite avec `BeanOntology`. Elle définit sept prédicats :

`Demande`, `Offre`, `Refus`, `ContreOffre`, `EtatCourse`, `Affectation` et `Paiement`. Un message porte ainsi un contenu typé, par exemple `((Offre :prix 16.0 :eta 8 ...))`, plutôt qu'une chaîne libre.

1. **Inscription.** Dans `setup()`, le taxi s'inscrit au DF sous `taxi-service`. Il s'en retire dès qu'il est occupé, en ravitaillement ou en panne. Avec un seul passager, il s'inscrit sous `taxi-covoiturage`.
2. **Recherche.** Le client interroge le DF et envoie un `CFP(Demande)` contenant sa position et sa destination.
3. **Réponse.** Chaque taxi répond `PROPOSE(Offre)` ou `REFUSE(Refus)`.
4. **Choix.** Le client envoie `ACCEPT_PROPOSAL` au taxi retenu et `REJECT_PROPOSAL` aux autres, ou bien une contre-offre `PROPOSE(ContreOffre)`.
5. **Engagement.** Le taxi confirme par `INFORM(EtatCourse en-route)`. Si un autre client l'a réservé entre-temps, il répond `FAILURE`, et le client relance.
6. **Course.** Le taxi envoie `INFORM` *pris-en-charge*, puis *arrive* avec le prix. Le client paie par `INFORM(Paiement)`.

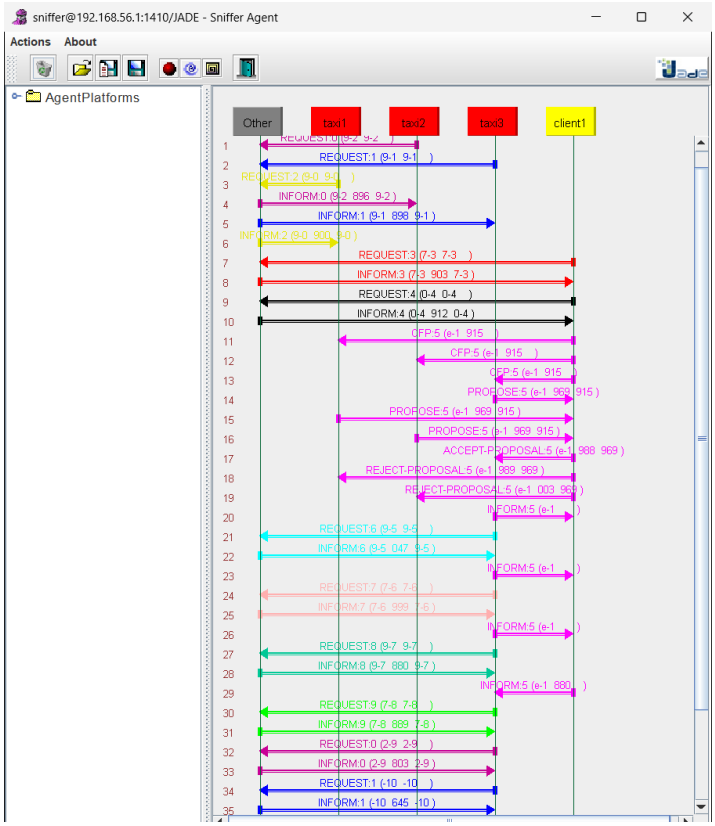


FIGURE 6 – Scénario nominal dans le Sniffer : inscriptions au DF (« Other »), CFP aux trois taxis, trois PROPOSE, un ACCEPT et deux REJECT, puis le premier INFORM de la course (en-route).

Mise en œuvre. Le lanceur `taxi.Lancement` utilise l'API `Runtime` de JADE pour créer trois conteneurs : *Main-Container*, *Conteneur-Taxis* et *Conteneur-Clients*. Un scénario `multijvm` fait aussi rejoindre la plateforme par des clients lancés depuis une seconde JVM. Une fenêtre Swing « Ville » affiche la grille, les états et les itinéraires. Elle sert à observer et à agir sur l'environnement (pause, vitesse, ajout d'un client par un clic), mais ne prend aucune décision à la place des agents. Pour comparer avec les classes prêtes à l'emploi, le paquet `taxi.fipa` refait le même protocole avec `ContractNetInitiator` et `ContractNetResponder`.

5 Scénarios et résultats

Chaque scénario se lance avec `.\run.ps1 <scenario>`. Les traces ci-dessous sont extraites de nos exécutions. Les heures et les `utilité=...` ont été retirés pour gagner de la place.

5.1 Scénario nominal : compromis prix / attente

Trois taxis répondent au même client (budget 30, attente maximale 9).

```
[taxi2] PROPOSE àclient1 : prix=26.00, attente=10
[taxi3] PROPOSE àclient1 : prix=16.00, attente=8
[taxi1] PROPOSE àclient1 : prix=21.00, attente=5
[client1] écarte taxi2 (attente trop longue)
[client1] CHOISIT taxi3 (prix=16.00, attente=8, coût perçu=20.00 selon profil normal)
[taxi3] ENGAGÉ pour client1 -> va le chercher en (3,2), arrivée prévue dans 8 ticks
[taxi3] retiré du DF (taxi-service)
[client1] pris en charge par taxi3 après 8 tick(s) d'attente
[client1] arrivé en (9,8), payé 16.00, budget restant 14.00
[taxi3] paiement reçu de client1 : 16.00 (gains totaux 16.00)
```

taxi1 arrive plus vite, mais son coût perçu est de $21 + 0{,}5 \times 5 = 23{,}5$, contre 20 pour taxi3 : le client accepte d'attendre 3 ticks de plus pour économiser 5. L'automate parcourt RELANCER → CHERCHER → CHOISIR → ATTENDRE → VOYAGER → ARRIVE.

5.2 Concurrence entre clients, et mode réservation

Deux clients acceptent taxi1 au même instant. Le premier ACCEPT reçu l'emporte, et l'autre client reçoit un FAILURE, puis relance :

```
[client1] CHOISIT taxi1 (prix=11.55, attente=2, ...)
[client2] CHOISIT taxi1 (prix=18.00, attente=4, ...)
[taxi1] ENGAGÉ pour client1 -> va le chercher en (1,1)
[taxi1] FAILURE pour client2 : vers_client
[client2] relance 1/5 dans 3000 ms
...
[client2] CHOISIT taxi1 (prix=21.60, attente=3 ...)
```

En mode `reservation`, le taxi garde sa première offre réservée pendant 2 s et refuse les autres CFP (`offre-en-cours`). Le conflit disparaît : il n'y a plus de FAILURE, mais le second client doit relancer plus tôt. Les deux variantes illustrent le compromis entre réactivité et cohérence des engagements.

5.3 Négociation et révision des désirs

```
[client1] écarte taxi1 (trop cher)
[client1] NÉGOCIE avec taxi1 : contre-offre 17.00 au lieu de 21.00
[taxi1] ACCEPTE la contre-offre de client1 : 17.00 au lieu de 21.00
[client2] NÉGOCIE avec taxi1 : contre-offre 20.00 au lieu de 27.00
[taxi1] REJETTE la contre-offre de client2 (20.00 < 80% de 27.00)
```

Dans le scénario `revision`, le seul taxi met 6 ticks à arriver, alors que le client n'accepte que 5 ticks d'attente. Après deux échecs, le client révise son désir (attente maximale 5 → 8) et accepte :

```
[client1] écarte taxi1 (attente trop longue) (x2)
[client1] RÉVISION DES DÉSIRS : attente max 5 -> 8, prix accepté jusqu'à 36.00 (budget +20%)
[client1] CHOISIT taxi1 (prix=13.50, attente=6, coût perçu=16.50 selon profil normal)
```

Dans le scénario `budget` (course à 39 pour un budget de 10), même la révision ne suffit pas, et le client abandonne (RELANCER → ABANDONNER).

5.4 Carburant, profils, covoiturage et trafic

```
[taxi1] REFUSE client1 : carburant (12 < 18)
[taxi1] carburant 12/40 -> part se ravitailler en (0,0)
[taxi1] plein fait (40/40)
[client2] CHOISIT taxi1 (prix=9.90, attente=3, ...)
```

Le taxi préfère refuser une course plutôt que risquer la panne : son désir de rester opérationnel l'emporte sur celui de gagner de l'argent. Une fois le plein fait, il se réinscrit au DF et sert le client suivant.

Profils. Avec les mêmes offres (taxi proche à 21 pour 2 ticks d'attente, taxi lointain à 8 pour 9 ticks), le client *pressé* choisit le taxi proche (coût perçu 25) et le client *économe* le taxi lointain (coût perçu 9,6).

Covoiturage. Après avoir pris client1, taxi1 s'inscrit sous `taxi-covoiturage` et propose une place à client2, qui se trouve sur sa route :

```
[taxi1] PROPOSE àclient2 : prix=8.09, attente=1 [COVOITURAGE -30%]
[taxi1] client client2 pris en charge en (5,1) (2 passagers àbord)
[taxi1] client client1 déposé en (9,1), àpayer : 15.00
[taxi1] client client2 déposé en (9,2), àpayer : 8.09
```

Trafic. Dans le scénario `trafic`, le taxi contourne le centre : *itinéraire de la course (Dijkstra) : 10 cases, 10 ticks (détour de 2 case(s) pour éviter les embouteillages).*

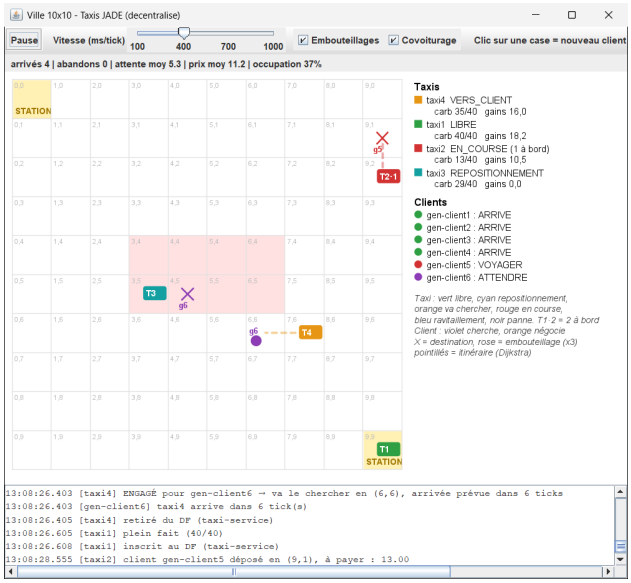


FIGURE 7 – Scénario `ville` : taxi4 va chercher un client (orange), taxi2 transporte un passager (rouge), taxi3 se repositionne (cyan) ; centre embouteillé en rose.

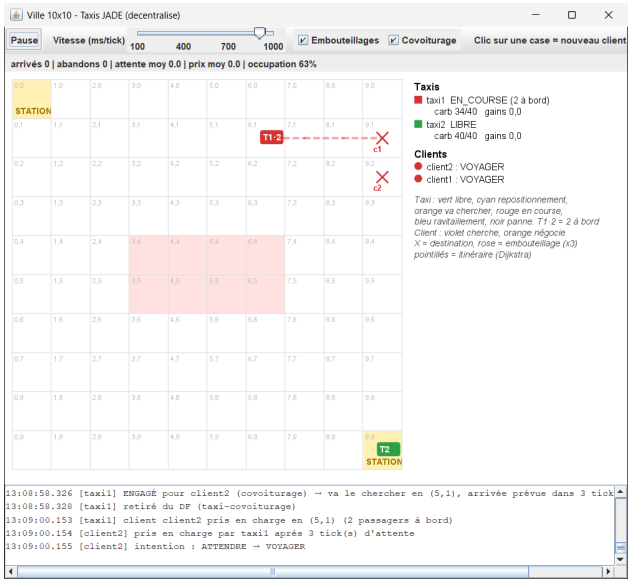


FIGURE 8 – Covoiturage : taxi1 transporte deux passagers (« T1 · 2 ») vers leurs destinations voisines c1 et c2.

5.5 Simulation continue et comparaison avec un dispatcher

Dans le scénario `ville`, un `GeneratorAgent` crée 20 clients aléatoires pour 4 taxis. Résultat : 20 clients arrivés, aucun abandon, une attente moyenne de 5,3 ticks, 5 négociations réussies, 3 covoiturages et 3 révisions de désirs.

Le script `experience.ps1` compare ensuite l'approche décentralisée (DF + Contract Net) à un dispatcher central qui affecte à chaque client le taxi le plus rapide. Les deux modes reçoivent les *mêmes* 20 clients (graine 42), avec 1, 2, 3 puis 5 taxis.

	Taux d'abandon		Attente moy. (ticks)		Occupation des taxis	
Taxis	Décentr.	Central	Décentr.	Central	Décentr.	Central
1	55 %	45 %	6,3	5,4	45 %	51 %
2	**10 %**	25 %	5,3	6,1	41 %	35 %
3	**0 %**	5 %	4,1	4,9	27 %	28 %
5	**0 %**	5 %	4,5	4,8	18 %	16 %

TABLE 2 – Comparaison décentralisé / centralisé (20 clients, graine 42, `results/experience.csv`).

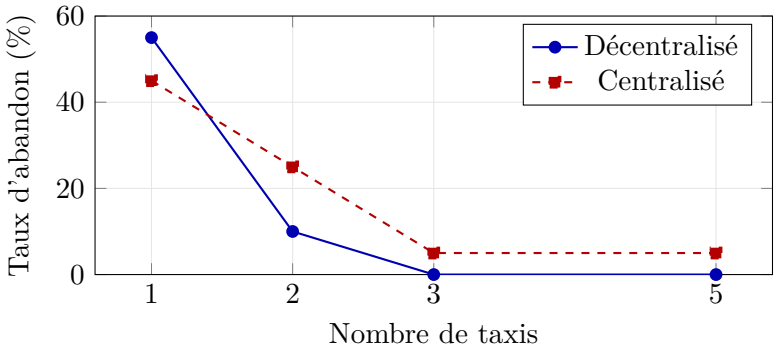


FIGURE 9 – Taux d'abandon selon la taille de la flotte.

Analyse. Avec un seul taxi, la flotte est saturée dans les deux cas : la demande dépasse l'offre, et le dispatcher fait un peu mieux grâce à sa vision globale. Dès deux taxis, l'approche décentralisée l'emporte nettement (10 % d'abandons contre 25 %). Le client y garde la main : il négocie (4 contre-offres acceptées), arbitre selon son profil et révise ses désirs. Le dispatcher, lui, applique un critère unique (le taxi le plus rapide) et ne négocie pas. Au-delà de trois taxis, la flotte est surdimensionnée : l'occupation tombe sous 20 %. Ces chiffres viennent d'une seule exécution par configuration et dépendent des temporisations : il faut les lire comme des tendances, pas comme des mesures définitives.

6 Conclusion

Ce TP nous a d'abord permis de prendre en main JADE : installation sous Windows et IntelliJ, rôle des agents système (AMS, DF, RMA), outils de débogage comme le Sniffer, puis écriture d'agents qui communiquent par messages ACL avec `createReply()`. Nous avons ensuite réalisé un système multi-agents BDI de réservation de taxis entièrement décentralisé : deux classes d'agents, le DF comme seul annuaire, et un protocole Contract Net dont les contenus sont typés par une ontologie SL.

Le modèle BDI se retrouve directement dans le code. Les croyances sont les attributs des agents. Les désirs sont des fonctions d'évaluation : le coût perçu pondéré par le profil pour le client, l'utilité sous contrainte de carburant pour le taxi. Les intentions sont un automate (`FSMBehaviour`) pour le client et un plan d'étapes pour le taxi. La révision des désirs après échec, la négociation et le covoiturage montrent que les agents adaptent leur comportement plutôt que de suivre un script figé. Les scénarios couvrent les cas nominaux comme les cas dégradés : concurrence, budget insuffisant, panne de carburant, aucun taxi disponible. La comparaison avec un dispatcher montre qu'à flotte suffisante, l'approche décentralisée sert mieux les clients.

Limites et perspectives.

— La délibération reste codée en Java. Une plateforme BDI dédiée (Jadex, ou JaCaMo/Jason au-dessus de JADE) donnerait des désirs et des plans déclaratifs.

— Les temps d'attente du protocole sont exprimés en temps réel, alors que la simulation avance en ticks. Une horloge simulée unique rendrait les expériences reproductibles à l'identique.

— Pistes d'extension : répéter chaque expérience plusieurs fois pour obtenir des moyennes et des écarts-types ; enchères entre taxis ; apprentissage des tarifs ; trafic dynamique ; déploiement réel sur plusieurs machines.